quy gán nguồn cho rag hai tầng

tập đủ tối tiểu + integrated gradients trên đồ thị

tầng a — tìm **tập đủ tối tiểu** trong pool ứng viên

mwc · deegan–packel · banzhaf có dấu

tầng b — tìm **đường** trên đồ thị tri thức

integrated gradients trên gate của cạnh

bài toán

rag trả lời đúng, nhưng không nói được *vì sao*. hai câu hỏi khác nhau:

đoạn nào gánh câu trả lời

trong n đoạn đã lấy về, tập nào *thật sự* đủ để trả lời, và không đoạn nào thừa? prompt phình theo số đoạn ⇒ ablation

vì sao truy hồi tìm ra chúng

trên đồ thị tri thức, *đường* nào dẫn từ thực thể trong câu hỏi tới đoạn đó? đồ thị cố định, hàng nghìn cạnh ⇒ ig

hai câu hỏi cần hai công cụ khác nhau. ghép chúng bằng **một giao diện duy nhất**: tầng a trả về tập đích T, tầng b nhận T. **không điểm số nào đi qua.**

toàn cảnh

```
truy vấn q
   |
   |  3 nguồn -> pool thô K <= 60 -> khử trùng lặp -> rerank THEO TỪNG NGUỒN
   v
pool C(q),  n = 40  =  20 bm25  +  15 gfm  +  5 ngẫu nhiên
   |
   |  >>> TẦNG A   tìm tập đủ tối tiểu            (năm bước a1..a5)
   |      v(S)     model đọc S rồi trả lời - khớp đáp án vàng không?      a1
   |      adq(S)   các đoạn trong S có THẬT SỰ nói ra đáp án không?       a2
   |      tau_q    ngưỡng riêng từng truy vấn, chốt từ trần quan sát      a3
   |      g_theta  ước lượng v(S) để khỏi gọi llm cho mọi tập con         a4
   |      tìm      đi lên từ nghiệm phủ, rồi chứng minh tối tiểu          a5
   v
T = S*  =  {đoạn 3, đoạn 12}          <- THỨ DUY NHẤT đi qua ranh giới
   |
   |  >>> TẦNG B   tìm đường trên đồ thị          (ba bước b1..b3)
   |      b1 ĐO    ig trên gate của từng (cạnh, tầng)
   |      b2 NỐI   trải phẳng theo tầng, quy hoạch động max-sum
   |      b3 THỬ   suff / nec bằng model thật
   v
Pi_D  =  {đường 1, đường 2}  +  suff(Pi_D)  +  credit từng đoạn
```

phương pháp tầng a — năm bước

	làm gì	bằng công cụ gì	giá mỗi truy vấn	cho ra
a1	đo *trả lời đúng*	llm sinh + llm judge	2 lượt mỗi *tập con*	$v(S) \in [0,1]$
a2	đo *có bằng chứng*	chẻ câu hỏi + model nli nhỏ	1 lượt, rồi 0	bảng phủ, $\mathrm{adq}(S)$
a3	chốt ngưỡng riêng cho truy vấn	$v(\emptyset)$, $v(R)$, $v(C)$	3 lượt	τ_q, v_{ref}
a4	thay v bằng ước lượng rẻ	surrogate g_θ	**0 lượt**	$\hat{v}(S)$
a5	tìm, rồi *chứng minh* tối tiểu	tầng phủ + beam + quét dàn	~ 6 lượt	$T = S^*$

hai chỗ quyết định toàn bộ chi phí. **a2** chạy *trước* **a1** dù nó kiểm điều kiện thứ hai — vì nó miễn phí và loại $\sim 45\%$ truy vấn ngay. **a4** tồn tại chỉ để 2^{40} tập con không phải đi qua llm.

· mọi slide sau đây mang nhãn bước ở góc phải tiêu đề

nền trò chơi hợp tác — bốn khái niệm, một ví dụ

ba đoạn a, b, c. $v(S)$ là điểm khi cho model đọc đúng tập S.

ví dụ chạy suốt phần này

```
v(rỗng)      0,05   ngưỡng tau_q = 0,84
v({a})       0,12
v({b})       0,18
v({c})       0,71   <- cao mà KHÔNG có dữ kiện
v({a,b})     0,94   <- bắc cầu
v({a,c})     0,74
v({b,c})     0,76
v({a,b,c})   0,95
```

tập thắng: $\{a, b\}$, $\{a, b, c\}$. **tối tiểu: chỉ** $\{a, b\}$ — $\{a, b, c\}$ thắng nhưng chứa một tập thắng nhỏ hơn.

bốn khái niệm dùng ngay ở slide sau

trò chơi	(N, v) — N là pool, v là hàm giá trị
đơn điệu	$S \subseteq S' \Rightarrow v(S) \leq v(S')$. **ta KHÔNG có tính chất này**
thắng	$v(S) \geq \tau_q$
mwc	thắng, và không tập con thật sự nào thắng

v không đơn điệu, và ta không sửa

ví dụ trên tình cờ đơn điệu; thực tế **không** — ta gặp đúng ca đó ở slide **tối tiểu** ($v(\{a\}) = 0{,}85$ mà $v(\{a, b\}) = 0{,}60$). định nghĩa mwc **không cần đơn điệu**, và bao đơn điệu $\bar{v}(S) = \max_{S' \subseteq S} v(S')$ cho $\text{mwc}(\bar{v}) = W$ — **không đổi gì ngoài chi phí.**

đối tượng cần tìm

$$S \text{ đủ} \iff \underbrace{v(S) \geq \tau_q}_{\text{trả lời đúng}} \wedge \underbrace{\mathrm{adq}(S)}_{\text{có bằng chứng}}$$

$$S \text{ tối tiểu} \iff S \text{ đủ} \wedge \nexists S' \subsetneq S \text{ đủ} \qquad W = \{\, S \text{ đủ tối tiểu} \,\} \qquad T = S^* \in W$$

dư thừa hiện ra thành bội số mwc

ba đoạn trùng ý ⇒ ba mwc khác nhau, mỗi tập chứa một đoạn.

nên bản này **không cần gom cụm để hạ** 2^K — dư thừa tự lộ trong cấu trúc W, còn 2^K thì surrogate **a4** lo.

cái giá: bỏ shapley là bỏ completeness

shapley có $\sum_c \varphi_c = F(E) - F(E')$ — một đẳng thức chính xác, bắt được mọi lỗi cài đặt.

mwc không có đẳng thức đó. đổi lại: được cả *họ* lời giải thích, chỉ số cấu trúc, và độ tin cậy. tầng b lấy lại đẳng thức này bằng c/h

«đủ» có hai thành phần, và *v* chỉ đo một

ca mồi tham số

```
q  ai đạo diễn phim thắng
   oscar 1994?
y  Robert Zemeckis

đoạn 19  "Forrest Gump là hiện
          tượng văn hóa thập
          niên 1990"
   -> KHÔNG nói đạo diễn
   -> KHÔNG nói thắng giải

model thấy tên phim
   -> TỰ NHỮ -> trả lời ĐÚNG
v({19}) = 0,71   cao
```

định nghĩa chỉ-dùng-*v* **nhận đoạn 19 vào** *T*, rồi dạy retriever đi lấy nó.

và lệch này đã được đo

model trả lời đúng khi ngữ cảnh không đủ

trên các bộ đo trong bài	35–62 %

trong các ca model trả lời sai

do *không có* ngữ cảnh	10,2 %
do ngữ cảnh *có mà không đủ*	66,1 %

sau truy hồi, tỉ lệ ca thật sự đủ ngữ cảnh

musique-ans	46,2 %
hotpotqa	44,6 %

joren et al., iclr 2025 — arXiv:2411.06037

hai bộ ta nhắm: hơn *nửa* truy vấn **không có tập đủ trong pool.** nên điều kiện thứ hai không phải cầu toàn — nó là ca *đa số*.

hai điều kiện, bốn ô — mỗi ô một nguyên nhân khác

	adq phủ hết	**adq không phủ**
$v \geq \tau_q$	**nhãn tốt** — nhận	**mồi tham số** — loại
$v < \tau_q$	bằng chứng *có* mà model không nối được → lỗi **model sinh**, tầng b **không sửa được**	pool *thiếu* → thoát sớm ở a0/c0

giữ adq **tách khỏi** v, không gộp thành một điểm số — gộp thì mất bảng này, và mất luôn khả năng tách *lỗi truy hồi* khỏi *lỗi model sinh*. ô dưới-trái là ô mà bản chỉ-dùng-v không bao giờ thấy.

$v(S)$ — hai lượt llm, một con số

bước 1 · sinh từ đúng S

```
Answer using ONLY the passages below.
If insufficient, say: INSUFFICIENT.

[1] <nội dung chunk thứ nhất trong S>
[2] <nội dung chunk thứ hai>
Question: <q>
Answer:
```

$$\hat{a}_S = \text{generate}(q, S)$$

bước 2 · judge, đọc logprob token đầu

```
Question: <q>
Gold answer: Robert Zemeckis
Also acceptable: Zemeckis · Robert Lee
Candidate answer: <â_S>
Correct? One word: Yes or No.
Answer:
```

$$v(S) = \frac{\sum_{t\in\mathcal{Y}} e^{\ell_t}}{\sum_{t\in\mathcal{Y}} e^{\ell_t} + \sum_{t\in\mathcal{N}} e^{\ell_t}} \in [0,1]$$

ℓ_t = logprob token đầu · $\mathcal{Y}$ = {Yes, yes, ...}

ba chỗ không được làm sai

không log-likelihood của y — *"Paris"* vs *"Paris, France"* đúng ngữ nghĩa mà logprob thấp, và lệch đó *không* triệt tiêu giữa các tập con: **thứ tự lật**.

không thang 1–5 — 5 mức làm hàng chục tập con hòa nhau, không phân biệt được S với $S \setminus \{c\}$, và *tối tiểu* mất nghĩa.

phải nhét `answer_aliases` vào prompt judge, không thì tập *đúng* bị loại oan.

$\text{cost}(S) = \text{overhead} + \sum_{k\in S} \text{len}(k)$ — không phải $500 + 250|S|$, vì document $\sim$100 và summary $\sim$600 là hai lớp độ dài

bước 1 · chẻ thành mảnh

hỏi ai: musique gán sẵn phân rã 1-hop + đáp án trung gian. corpus riêng thì llm, **1 lượt mỗi truy vấn** — không phải mỗi tập con.

```
mảnh 1  phim thắng oscar 1994 là
        Forrest Gump
mảnh 2  Forrest Gump do Zemeckis
        đạo diễn
```

bước 2 · đối chiếu

hỏi ai: model nli nhỏ (∼400M), chạy local, **0 lượt api**.

$$e(k, j) = \mathrm{nli}(\text{đoạn } k,\ \text{mảnh } j)$$

đoạn k vào chỗ *premise*, mảnh j vào chỗ *hypothesis* · $40 \times J$ lượt forward model nhỏ

bảng phủ — 0 lượt llm

	mảnh 1	mảnh 2	
đoạn 3	entail	-	
đoạn 7	entail	-	
đoạn 12	-	entail	
đoạn 23	-	entail	
đoạn 31	-	entail	
đoạn 19	-	-	<- bẫy

$$\mathrm{cov}(S) = \{\, j : \exists k \in S,\ e(k, j) = \text{entail} \,\}$$

$$\mathrm{adq}(S) \iff \mathrm{cov}(S) = \{1, \ldots, J\}$$

bảng phủ cho **ba** thứ, cả ba miễn phí:

```
1 CỬA THOÁT  mảnh nào trống -> bỏ ngay
2 ỨNG VIÊN   phủ tập nhỏ, giải CHÍNH XÁC
             {3,12} {3,23} {7,12} ...
3 PRIOR |T|  |T| nên bằng J
```

vì sao phải chẻ ra — hai lý do độc lập

độ dài

premise = cả S có thể tới $10{,}000$ token, mà model nli chỉ nhận 512.

chẻ ra thì mỗi lượt kiểm là **1 đoạn** ($\sim$100 token) đối **1 câu ngắn**.

bắc cầu

claim ghép *"phim thắng 1994 do Zemeckis đạo diễn"* thì **không đoạn nào một mình** khẳng định được.

không chẻ $\Rightarrow$ loại oan cả $\{3, 12\}$, tức loại đúng **ca multi-hop** mà cả dự án nhắm.

chẻ ra thì mỗi mảnh có *đúng một* đoạn khẳng định nó, và ghép lại thành T đầy đủ. · cơ chế

theo arXiv:2608.00585 — kiểm từng đoạn đối câu hỏi gốc thất bại ở multi-hop

$$\tau_q = \theta \left[v_{\text{ref}}(q) - v(\emptyset) \right] + v(\emptyset), \qquad \theta = 0{,}95$$

$$v_{\text{ref}}(q) = \max\Big(v(C),\ v(R),\ \max_k v(\{k\}),\ \text{mọi } v(S) \text{ đã quan sát} \Big)$$

mốc phải là cực đại quan sát được

khi C còn đoạn nhiễu thì một tập con có thể ghi điểm *cao hơn* cả C — neo vào $v(C)$ là neo vào **mốc bị dìm**.

```
v(C)  = 0,81   nhồi 40 đoạn -> phân tán
v(R)  = 0,88   12 đoạn, ít nhiễu hơn
->  v_ref = 0,88,  tau_q = 0,84
```

hai mốc đặc biệt

```
v(rỗng)   không cho đoạn nào
          -> chỉ tri thức tham số
v(R)      12 đoạn: top-2 mỗi cụm, VÀ
          tôn trọng hạn ngạch 20/15/5
```

chốt chặn mẫu số nhỏ: $v_{\text{ref}} - v(\emptyset) < 0{,}05 \Rightarrow T = \emptyset$, cờ *"truy hồi không đóng góp"* — ca có thật, không phải lỗi.

cập nhật v_{ref} tự nhất quán — và $\theta < 1$ gánh tất

quan sát mới có thể *cao hơn* trần đang dùng. lúc đó v_{ref} tăng, nên τ_q cũng tăng — liệu tập vừa nâng trần có bị chính ngưỡng mới loại?

$$v(S) > v_{\text{ref}} \implies v_{\text{ref}} := v(S) \implies \tau_q' = \theta\big[v(S) - v(\emptyset)\big] + v(\emptyset) \underbrace{<}_{\theta < 1} v(S)$$

không bao giờ tự phủ định

tập vừa nâng trần **luôn vẫn vượt ngưỡng mới**. cập nhật không thể loại chính cái gây ra nó.

không phải xét lại lịch sử

τ_q chỉ *tăng*, nên mọi tập đã bị loại vẫn bị loại.

$\theta = 1$ thì suy biến: $\tau_q' = v(S)$, tập nâng trần chỉ *bằng* ngưỡng, và tính chất trên mất

surrogate g_θ — một hàm, hai pha

$$g_\theta\Big(q,\ S,\ \{\hat{s}(q,k)\}_{k\in S},\ \{\tilde{v}_k\}_{k\in S},\ v(\emptyset)\Big)\ \longrightarrow\ \hat{v}(S)$$

bất biến hoán vị trên S · $\hat{s}$ = điểm rerank, luôn có · $\tilde{v}_k = v(\{k\}) - v(\emptyset)$, chỉ khi đã trả vòng 0

τ_q không được là đầu vào

g_θ hồi quy lên $v(S)$; so với ngưỡng xảy ra *sau*, bên ngoài mô hình.

bước c2 gọi g_θ để quyết định có mua $v(C)$ — mà $v(C)$ mới chốt v_{ref} rồi chốt τ_q. cho τ_q vào đầu vào là **tạo vòng tròn thật**.

τ_q xuất hiện trong *mất mát*, không ở đầu vào:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda\mathcal{L}_{\text{rank}}$$

hinge theo cặp *trong một* truy vấn

dữ liệu — là log của chính các lượt gọi v

nguồn	truy vấn	nhãn	tổng
pha 1, bốn vòng	10.000	x111	1.110.000
verification pha 2	188.000	x 10	1.880.000
tỉa, $\|S*\|$ = 6..10	8.000	x 13	104.000
cứu -> pipeline đầy đủ	7.400	x111	821.000
trượt bị bỏ	14.000	x 5	70.000
			3.983.000

$|S|$ chạy từ 0 tới 40 — **không cỡ nào đặc biệt**, nên *một* mô hình, không phải một cho singleton và một cho phần còn lại.

che $\tilde{v}_k$ theo từng chunk, và nghiệm thu

ba khuôn, vì thực tế có ba chế độ

```
pha 1 + cứu          cả 40 chunk có v~_k
verification pha 2 CHỈ |T| chunk có
tìm theo tầng (c5) không chunk nào có
```

trộn ba khuôn theo đúng tỉ lệ chúng xuất hiện.

che tất-cả-hoặc-không-gì **không dạy được khuôn giữa** — mà đó là khuôn của *mọi* lượt verification ở pha 2.

nghiệm thu, và sàn phải vượt

auc trong từng truy vấn, rồi trung bình qua các truy vấn.

sàn: mô hình chỉ nhìn $|S|$.

auc *gộp* là bẫy: tập lớn thì v cao nên con số đẹp sẵn, mà phân biệt *trong* truy vấn bằng **0** — mọi tập cỡ 3 hòa điểm, beam thành chọn bừa.

mốc kiến trúc phải vượt

gbm trên đặc trưng tay chạy **trước**. set transformer chỉ đáng chọn nếu vượt nó trên auc-trong-truy-vấn.

tìm theo tầng, đi lên từ nghiệm phủ

vì sao chỉ tìm trong vùng này

```
tập KHÔNG phủ hết mảnh
  -> nếu vượt tau_q thì chỉ có thể
     vì mồi tham số
  -> mà ta đã quyết LOẠI ca đó
  -> tìm ở đó là tìm thứ ta sẽ bỏ
```

cả dàn 2^n co lại thành **nghiệm phủ, và mọi tập chứa một nghiệm phủ**.

dự bị

bảng phủ có thể thiếu cột (nli bỏ sót bằng chứng diễn đạt lại). mọi tầng trượt $\Rightarrow$ quay về beam mò trên toàn dàn, tỉa theo r_i tăng dần.

các tầng

```
tầng 0  nghiệm phủ tối tiểu       |T| = J
        giải CHÍNH XÁC từ bảng phủ
tầng 1  nghiệm phủ + 1 đoạn
tầng 2  nghiệm phủ + 2 đoạn
...
mỗi tầng: chấm bằng g_theta, lấy top-beam,
          đo v theo thứ tự đó
```

hai lợi ích rơi ra từ cấu trúc

1 mọi ứng viên đã đủ về bằng chứng — không còn ứng viên mồi tham số.

2 thứ tự theo *cỡ*, nên tập nhỏ luôn thử trước: **"tối tiểu trước" là miễn phí theo cấu trúc**, không phải một bước tỉa riêng.

bỏ-một-phần-tử là SAI, vì v không đơn điệu

```
tau_q = 0,76
v({a})     = 0,85  >= tau_q  <- {a} ĐÃ ĐỦ
v({a,b})   = 0,60  <  tau_q     b làm lạc
v({a,c})   = 0,71  <  tau_q
v({b,c})   = 0,40  <  tau_q
v({a,b,c}) = 0,80  >= tau_q

bỏ-một-phần-tử:  ba cặp đều trượt
                 -> tuyên "tối tiểu"  SAI
sự thật:         {a} thắng -> thừa b, c
```

nhưng chỉ tập con CÒN PHỦ mới đe dọa

```
T = {3, 12, 19}
  3 phủ mảnh 1 · 12 phủ mảnh 2 · 19 thừa

tập con thực sự:         7 tập
trong đó CÒN phủ hết:    1 tập -- {3,12}
  {3,19} mất mảnh 2 · {12,19} mất mảnh 1
  mọi đơn tử mất một mảnh
```

và tầng 0 **đã đo** $\{3, 12\}$ **rồi** — nên tính tối tiểu **không tốn lượt nào thêm**.

khi không có bảng phủ: $2^{|S^*|} - 2$ lượt

$\lvert S^* \rvert$	3	5	6	8	10
quét hết	\$0,02	\$0,11	\$0,24	\$1,14	\$5,37
tỉa rồi quét	—	\$0,11	\$0,04	\$0,07	\$0,10

ba chỉ số trò chơi — tính thẳng trên ví dụ a, b, c

$$\beta_i^+ = \big|\{\, S \subseteq N \backslash \{i\} : v(S) < \tau_q \wedge v(S \cup \{i\}) \geq \tau_q \,\}\big| \qquad \beta_i = \frac{\beta_i^+ - \beta_i^-}{2^{\,n-1}} \qquad \mathrm{pgi}_i = \big|\{\, S \in W : i \in S \}\big|$$

β_i^+: thêm i biến thiếu → đủ · β_i^-: biến đủ → thiếu, $= 0$ nếu đơn điệu · pgi chỉ đếm trên các mwc

đếm tay, $\tau_q = 0{,}84$, $n = 3$

```
i = a  {b}   0,18 -> {a,b} 0,94  pivotal
       {b,c} 0,76 -> 0,95        pivotal
       hai ca kia: không           B+ = 2
i = b  {a}   0,12 -> {a,b} 0,94  pivotal
       {a,c} 0,74 -> 0,95        pivotal
       hai ca kia: không           B+ = 2
i = c  {a,b} đã thắng -> KHÔNG tính
       ba ca kia: vẫn thua         B+ = 0
```

kết quả

	a	b	c
$v(\{i\})$	0,12	0,18	**0,71**
β_i	0,50	0,50	**0**
pgi_i	1	1	**0**

đây chính là điều cần thấy

c có *điểm đơn lẻ cao nhất* $(0{,}71)$ mà **cả hai chỉ số đều cho 0** — vì *c* **không bao giờ** là thứ biến một tập thiếu thành đủ.

chỉ số cấu trúc, và đầu ra

$$\mathrm{ov}(S) = \sum_{S' \neq S} \frac{|S \cap S'|}{|S \cup S'|}$$

độ trùng lặp giữa các mwc — cao nghĩa là nhiều lời giải thích gần như nhau

$\beta_i < 0$ chỉ thẳng ra đoạn gây hại

đúng vai mà $\varphi_c < 0$ đóng ở bản shapley: **không đơn điệu** ở đây là **thông tin**, không phải khuyết tật.

chuẩn hóa β và pgi riêng rồi mới gộp — $\beta \in [-1, 1]$ còn pgi đếm tới hàng chục, cộng thẳng thì pgi nuốt β.

chọn S^*

```
parsimony    -|S|
structure    sigma_g = w_b*beta + w_p*PGI
rank         tổng r_i   (chỉ phá hòa cuối)
-> lọc Pareto, rồi gộp có trọng số
```

đầu ra

```
T = S*                <- THỨ DUY NHẤT qua ranh giới
+ phiên bản g_theta    lô nào gán
+ đường đi             qua · tỉa · thử lại · cứu
+ trọng số             N_s / n_s
+ v(T) - tau_q         khoảng vượt ngưỡng
+ cov(T), J            phủ mấy trên mấy mảnh

W, p(S), entropy(p)   báo cáo, độ tin cậy
beta_i < 0            ứng viên V_neg (tầng b)
```

entropy thấp = một người thắng rõ · cao = nhiều mwc suýt soát

bài toán tầng b — thứ cần đọc ra là một CHUỖI

T từ tầng a, và đồ thị. gfm truyền tin L bước từ node khởi tới T. **câu hỏi:** *đường* nào dẫn tới $D \in T$?

vì sao không chỉ chấm điểm từng cạnh

điểm từng cạnh cho một *đống* cạnh quan trọng, không cho một *chuỗi*.

mà lời giải thích đọc được phải là

```
q -> Forrest Gump -> Zemeckis -> D
```

tức một **dãy cạnh nối tiếp**, mỗi bước ở đúng tầng của nó.

ba bước — xương sống của cả phần này

```
b1 ĐO   ig trên gate của từng (cạnh, tầng)
        -> cạnh nào gánh, ở bước nào

b2 NỐI  trải phẳng đồ thị theo tầng rồi
        quy hoạch động max-sum
        -> dãy cạnh nối tiếp, không phải đống

b3 THỬ  suff / nec bằng model THẬT
        -> đường tìm được có thật đủ không
```

điểm mục tiêu

$$F_B = \log p(v_D)$$

$p(v_D)$: xác suất gfm gán cho node đích D · một lượt forward cho ra $p(v)$ của *mọi* node, nên F_B của cả T dùng chung một lượt

integrated gradients — công cụ của bước b1

gradient tại một điểm chỉ nói độ dốc *ở đó*. ig tích phân dọc đường từ mốc tới điểm thật:

$$\mathrm{ig}_i \;=\; (x_i - x_i^0) \times \frac{1}{M} \sum_{m=1}^{M} \frac{\partial F\big(x^0 + \alpha_m (x - x^0)\big)}{\partial x_i}, \qquad \alpha_m = \tfrac{m}{M}$$

tính chất ta dùng

completeness $\sum_i \mathrm{ig}_i = F(x) - F(x^0)$

tổng đóng góp *bằng đúng* phần mà đầu vào tạo ra. nên credit chia được, và kiểm được bằng một đẳng thức.

đây là đẳng thức mà tầng a đã mất khi bỏ shapley — tầng b lấy lại được

vì sao ig cho cạnh, ablation cho đoạn

ig rẻ hơn ablation $\iff N > 3M$

```
cạnh đồ thị   hàng nghìn, đồ thị CỐ ĐỊNH -> ig
token         hàng trăm, prompt CỐ ĐỊNH  -> ig
đoạn / pool   prompt PHÌNH theo số đoạn
              -> tỉ lệ = 3M, K triệt tiêu
              -> KHÔNG có điểm hòa vốn
              -> ablation
```

gate trên cạnh — và phải riêng từng tầng

truyền tin bước l : $$h_v^{(l)} \;=\; \mathrm{agg}\Big(\big\{\, g_e^{(l)} \cdot m_e^{(l)} \;:\; e=(u\to v) \,\big\}\Big)$$

$g_e^{(l)} \in [0,1]$ là *gate* ta chèn vào cạnh e ở tầng l · $g \equiv 1$ cho đúng mô hình gốc

một gate cho mỗi cạnh là KHÔNG đủ

cùng một cạnh có thể cần ở bước 1 mà không cần ở bước 3. gate dùng chung **trộn hai vai** và ig trả về một số vô nghĩa.

```
gate chung        g_e          |E| biến
gate theo tầng    g_e^(l)      |E| x L biến
```

chỉ gate theo tầng mới đọc ra được *thứ tự* các bước — tức mới ra được một **chuỗi**.

ig trên gate

$$\mathrm{ig}_e^{(l)} = \big(g_e^{*(l)} - 0\big) \times \frac{1}{M}\sum_{m=1}^{M} \frac{\partial F_B\big(\alpha_m g^*\big)}{\partial g_e^{(l)}}$$

```
g* = 1     khi ĐÀO NHÃN
g* = ĝ     khi BÁO CÁO
```

hệ số $\hat{g}$ cắt cả hai chiều: tốt cho báo cáo trung thực, **tệ cho đào nhãn** — $\hat{g}$ là thứ model *làm*, không phải thứ ta dùng để *đo*. nên lúc mining luôn $g^* = 1$.

một đường π dùng h cạnh, và đường đó đóng góp c vào F_B. completeness của ig cho:

$$\sum_{e \in \pi} \mathrm{ig}_e = c \qquad \text{và nếu các cạnh đối xứng trong } \pi \text{ thì} \qquad \mathrm{ig}_e = \frac{c}{h}$$

vì sao đáng tiền

đây là một **đẳng thức kiểm được**, không phải một trực giác.

cài sai chỗ nào trong khâu ig — sai dấu, sai số bước M, sai mốc, nhân hai lần hệ số — thì $\sum_{e \in \pi} \mathrm{ig}_e \neq c$ và ta thấy ngay.

đúng vai mà $\sum_c \varphi_c = F(E) - F(E')$ đóng ở bản shapley — phần bất biến mà tầng a đã mất khi bỏ completeness

ca một đường, ba cạnh

```
c = 0,81          đóng góp của cả đường

ig cạnh 1 = 0,27
ig cạnh 2 = 0,27      mỗi cạnh = c/h
ig cạnh 3 = 0,27           = 0,81/3
            -----
tổng      = 0,81  = c        KHỚP
```

lệch $\Rightarrow$ có lỗi cài đặt, **dừng, đừng đi tiếp**.

vẽ lại: mỗi tầng một bản sao

```
node (v, l)   = node v ở bước l
cạnh (u,l) -> (v,l+1)  nếu u->v trong G

L tầng  ->  đồ thị KHÔNG CÓ CHU TRÌNH
        ->  quy hoạch động một lượt
```

đồ thị gốc có chu trình, nên không duyệt một lượt được. trải theo tầng thì **thứ tự topo là chính** l.

một lượt quét

$$\text{best}(v,l) = \max_{u\,:\,(u,v)\in E} \left[\text{best}(u,l-1) + \text{ig}^{(l)}_{(u,v)} \right]$$

lưu back$[v,l]$ để truy ngược ra dãy cạnh · $O((|V|+|E|)\,L)$

max-SUM, không phải min-cost

quen tay thì đặt cost $= -\log(\cdot)$ rồi chạy dijkstra. **sai ở đây.**

```
ig CỘNG TÍNH theo completeness:
   tổng ig trên một đường = đóng góp
   của đường đó

-log thì BIẾN TỔNG thành TÍCH
   -> mất completeness
   -> c/h không còn đúng
   -> mất luôn phép kiểm nội bộ
```

và ig *có dấu*: cạnh gây hại cho ig < 0, mà $-\log$ của số âm thì vô nghĩa.

kiểm chứng — và hai đại lượng khác nhau

$$\mathrm{suff}(\Pi_D) \;=\; \frac{\sum_{(e,l)\,\in\,\Pi_D} \mathrm{ig}_e^{(l)}}{\sum_{(e,l)} \mathrm{ig}_e^{(l)}} \qquad\qquad \mathrm{nec}(\pi) \;=\; F_B(g^*) - F_B\big(g^{*}\ \text{với}\ \pi\ \text{bị xóa}\big)$$

suff: phần của «cái bánh» mà tập đường đã lấy · nec: mất bao nhiêu nếu bỏ đúng đường đó · suff còn là *điều kiện dừng* của bước b2

suff tính trên TOÀN tập đường

không cộng suff của từng đường rồi lấy tổng — hai đường dùng chung cạnh thì cạnh đó bị đếm hai lần.

lấy **hợp các cạnh** rồi mới chia:

```
hợp cạnh: 0,22 + 0,19 + 0,33 = 0,74
          0,74 / 0,81 = 91%
suff = 0,91      đủ
```

suff có thể bị chặn dưới 1 — credit mồ côi

node *không* tới được từ bất kỳ seed nào vẫn góp vào **mẫu số**. nên suff thấp có *hai* nguyên nhân:

```
suff_max  tính MIỄN PHÍ từ bảng qhđ
          = trần mà mọi đường từ seed
            đạt được

suff_max < theta_s  -> entity linking
                       hỏng, dừng ngay
suff_max >= theta_s -> đồ thị thiếu cạnh
                       (ô «lỗi đồ thị»)
```

nhiều seed hội tụ — greedy xóa cạnh, rồi tỉa ngược

hai chặng

```
-- chặng 1: gom -------------------------
Pi_D <- rỗng
lặp:
    pi   = truy ngược back[] từ (D, L)
    gain = tổng ig các cạnh MỚI trong pi
    Pi_D <- Pi_D + pi

    suff(Pi_D) >= theta_s -> DỪNG: thành công
    gain < eps            -> DỪNG: bằng chứng
                             phân tán
    |Pi_D| = n_max        -> DỪNG: hết ngân
                             sách đọc hiểu

    đặt ig = 0 cho mọi cạnh đã lấy
    chạy lại qhđ

-- chặng 2: tỉa ngược -------------------
cho mỗi pi trong Pi_D (theo thứ tự thêm):
    suff(Pi_D \ {pi}) >= theta_s
        -> Pi_D <- Pi_D \ {pi}   đường thừa
```

cạnh dùng chung tự thành miễn phí

đặt $\text{ig} = 0$ cho cạnh đã lấy, nên vòng sau **không trả tiền lại** cho cạnh đó. hội tụ nhiều seed xử lý *tự nhiên*, không cần luật riêng.

$n_{\max}$ không phải hằng số chọn tay

$$n_{\max} = \left\lceil \frac{\theta_s}{\tilde{\varepsilon}} \right\rceil$$

$\tilde{\varepsilon}$: ngưỡng gain tương đối. mỗi đường được nhận phải góp $\geq \tilde{\varepsilon}$, nên quá $\theta_s/\tilde{\varepsilon}$ đường là không thể còn đủ

không phải $|V_q| + 1$ — số seed không bằng số đường độc lập.

chỉ một thứ đi qua ranh giới

giao diện

```
TẦNG A  ->  T = {D_2, D_7, D_15}  ->  TẦNG B

KHÔNG qua: v(S) · tau_q · beta_i · PGI
           g_theta · W · entropy
```

điểm của tầng a đo trên *pool*; tầng b định tuyến trên *đồ thị* — hai không gian khác nhau. và nhãn $(q \to T)$ **không phụ thuộc kiến trúc gfm**: đào một lần, dùng cho mọi lần train.

ba hàm mất mát

$$\mathcal{L}_{\text{node}} = \text{bce}\big(p(v),\ \mathbf{1}[v \in T]\big)$$

$$\mathcal{L}_{\text{hop}} = \sum_{l} \sum_{v \in V_{\text{hop}}(l)} \text{bce}\big(p^{(l)}(v),\ \mathbf{1}[(v,l) \in \Pi]\big)$$

$$\mathcal{L}_{\text{edge}} = \sum_{(e,l) \in \Pi} -\log \hat{g}_e^{(l)}$$

```
V_hop(l) = { v : (v,l) thuộc Pi }   dương
         + { v : (v,l') thuộc Pi }  âm KHÓ
                đúng node, SAI tầng
         + V_neg                    âm thường
```

không có $V_{\text{hop}}(l)$ thì tỉ lệ dương:âm thành $1 : 10^6$

bảng chẩn đoán — và nó cũng là nguồn nhãn

$$k_{\text{top}}(q) = \left\lceil \beta \,\middle|\, T(q) \,\middle|\, \right\rceil, \quad \beta \approx 1{,}5 \qquad\qquad \text{rank}_{\text{gfm}}(D) \text{ đọc từ } \textit{một} \text{ lượt forward}$$

ngưỡng hạng theo *từng ví dụ*, không phải một k cố định toàn cục

	rank **tốt**	rank **kém**
suff **cao**	**lành** — truy hồi tìm đúng, và đường giải thích được	**lỗi model** — đường có đó mà gfm không xếp D lên cao
suff **thấp**	**đường tắt** — xếp đúng *nhờ* lối khác, không nhờ bằng chứng	**lỗi đồ thị** — thiếu cạnh, hoặc entity linking hỏng

suff kiểm **đồ thị**, rank kiểm **model** — hai trục độc lập. ô *đường tắt* là ô dễ bỏ sót nhất: chỉ số đẹp mà lời giải thích rỗng. và mỗi ô **tự nó là một lô nhãn** để train vòng sau.

hai pha — mô hình ở pha 1, nhãn ở pha 2

chạy pipeline đầy đủ cho $200{,}000$ truy vấn thì không trả nổi. nên tách hai pha có *mục đích* khác nhau:

pha 1 · $N_1 = 10{,}000$ · llm đầy đủ

```
A0  chẻ câu hỏi + bảng phủ          400
A1  vòng 0  v(rỗng)·v(R)·v(C)
            + 40 singleton       44.500
A2  vòng 1  greedy biên          27.000
A3  vòng 2  active learning     ~37.000
A4  vòng 3  xác nhận            ~15.000
                                ---------
                                ~123.900 token
```

sản phẩm là g_θ, **không phải** nhãn cho tầng b.

pha 2 · $N_2 = 200{,}000$ · đường rẻ

```
C0  chẻ + bảng phủ                  400
      thiếu mảnh -> THOÁT SỚM
C1  v(rỗng)                         500
C2  g_theta chấm 40 đoạn đơn          0
C3  v(R)                          3.500
C4  v(C)                         10.500
      v_ref = max(v(R), v(C)); tau_q chốt
C5  tìm theo tầng bằng g_theta        0
C6  verification bằng llm THẬT  ~7.400
C7  ba nhánh (slide sau)
                                 --------
                                 ~22.300 token
```

c2 phải đứng trước c5 — hợp lệ vì g_θ không nhận τ_q

ba nhánh

```
(1) v(T) >= tau_q, adq phủ hết, tối tiểu
    -> XUẤT T sang tầng b

(2) v(T) < tau_q        T KHÔNG đủ
    -> thử ứng viên kế của beam, tối đa 3
       mỗi lần CHỈ 1 lượt (hết đủ thì dừng,
       không kiểm tối tiểu)
    -> qua: nhánh (1)
       3 lần trượt: nhánh (3)

(3) |T| >= 11 · tỉa kẹt ở >= 6 · thử hết
    -> HÀNG CỨU có phân tầng
```

nhánh 2 rẻ hơn tưởng

một ca trượt chỉ tốn **1 lượt** $(1{,}250$ token), một lượt cứu tốn $113{,}000$ — **90 lần thử bằng giá một lần cứu**. với 30% qua mỗi lần, 3 lần vớt $1 - 0{,}7^3 = 66\%$ số ca: p hiệu dụng $0{,}75 \to 0{,}91$.

«đủ mà không tối tiểu» KHÔNG vào hàng cứu

c6 đo *trọn* dàn tập con dưới T, nên tối tiểu đọc ra ngay, 0 lượt thêm:

```
v({a,b,c}) = 0,80 >= tau_q  T đủ
v({a,b})   = 0,78 >= tau_q  con cũng đủ
                      -> T KHÔNG tối tiểu
v({a}) = 0,30  v({b}) = 0,22
                      -> T = {a,b}
```

ca này xảy ra *thường xuyên*, vì lỗi hay gặp nhất của g_θ là đề xuất tập **phình**.

cứu là gì

chạy a1–a4 lên truy vấn đó. $v(\emptyset), v(R), v(C)$ đã có trong cache nên còn $113{,}000$ token.

τ_q *tăng* khi cứu — pha 2 vốn đang đo trần thiếu.

năm cổng — và chúng nằm trước 75% chi phí

	giai đoạn	chi phí	tích lũy	cổng chặn sau đó
0	hạ tầng · index test có sẵn	$0	$0	
0.5	chạy thử bốn cổng trên test index	$18	$18	**cổng 0.5** — chặn $1.100
1	hiệu chuẩn chuẩn ngoài	$7	$25	**cổng 1** — adq đáng tin?
2.0	index phần pha 1	$74	$99	
2	pha 1 đào nhãn	$103	$202	
3	surrogate	$0	$202	**cổng 2** — vượt gbm?
4	cổng p trên N_1 đủ	$0	$202	**cổng 3** — chặn $918
5.0	index phần pha 2	$815	$1.017	
5	pha 2 + cứu	$103	$1.120	**cổng 4** $\times 10$ lô

tiêu trước **cổng 0.5** là $18; tiêu trước **cổng 3** là $202. nếu thiết kế sai hoặc p thấp thì **không bao giờ trả $918 còn lại.**

sinh chạy local (qwen3-8b) nên $0 tiền, nhưng ~ 53 giờ gpu ở 20 sinh/giây — ràng buộc thật là **giờ gpu**, không phải tiền. $800+$ của bảng trên là OpenIE dựng index cho train, vì index dựng sẵn chỉ có cho *test* set.

những gì chưa chắc — và cổng nào đo nó

con số	trạng thái	ai đo
độ dài đoạn thật	giả định 250 tok	`0.b` — sai thì **mọi** bảng chi phí sai, và **không cổng nào bắt được**
p (tỉ lệ qua verification)	giả định $0{,}75$	cổng `0.5` đo rẻ, cổng `3` đo thật
45% thoát ở a0/c0	suy từ musique	`2.1` — và chắc chắn *lạc quan*, vì ta cố ý dùng checkpoint chưa fine-tune
adq sai bao nhiêu	chưa biết	cổng `1`, đối chiếu `supporting_documents`
cỡ rò rỉ nhãn vàng	chưa biết	`1.3` — chênh giữa pool chỉ-bm25 và pool đầy đủ

hai cổng **miễn phí** phải chạy trước khi cam kết ngân sách: **cổng** p (500 truy vấn pha 1 giữ lại, 0 lượt llm) và **chuẩn ngoài** (500 truy vấn test, \$7). không cổng nào được bỏ qua để «chạy nhanh hơn» — chúng rẻ *chính vì* nằm trước chi phí.

tóm lại

ba quyết định thiết kế

1 «đủ» có *hai* thành phần. *v* đo *trả lời đúng*, adq đo *có bằng chứng*. chỉ dùng *v* thì nhận cả ca mồi tham số.

2 tối tiểu, không phải điểm số. mwc cho cả *họ* lời giải thích; cái giá là mất completeness.

3 chỉ *T* qua ranh giới. hai tầng đo trên hai không gian khác nhau, nên điểm không mang qua được.

hai thứ còn phải chứng minh

p — tỉ lệ truy vấn mà đường rẻ qua được verification. cả bảng chi phí treo vào nó, và nó **chưa được đo**.

adq — model nli loại đúng hay loại bừa. chưa biết, và nó đang được giao quyền bỏ $\sim 45\%$ truy vấn.

bước tiếp theo

`0.a`–`0.j` trên test index (đã dựng sẵn, \$0), rồi cổng `0.5` với \$18.

github.com/pqvu0307/minimal_set_rag-graph_learning